แบ่งงาน

[วี] คนที่ 1: Data Engineer & Vector Master (เน้นขั้นตอนที่ 1-6: Offline Pipeline)

เสร็จภายในวันที่ 3 ส่งงานต่อเพื่อน

คนนี้จะเป็นคนคุม "คลังความรู้" เพื่อเตรียมข้อมูลให้ "ค้นได้"

- **Collect & Parse:** รวบรวมรีวิวหนังไทยจาก IMDB (อาจจะใช้ Dataset สำเร็จรูปหรือเขียน Script ดึงข้อมูล) และแยก Metadata เช่น ชื่อหนัง, ปีที่ฉาย, คะแนน
- **Data Cleaning:** ทำความสะอาดข้อความ ลบสัญลักษณ์ที่ไม่ได้ใช้ และจัดการปัญหาเรื่องภาษาไทย
- **Chunking & Embedding:** ออกแบบการแบ่งข้อความ (Chunk size) และเลือกใช้ Model สำหรับสร้าง Vector ที่รองรับภาษาไทยได้ดี
- **Vector DB:** ติดตั้งและจัดการฐานข้อมูล Vector (เช่น ChromaDB หรือ Pinecone) เพื่อเก็บข้อมูลพร้อมค้นหา

[อ๊อบ]คนที่ 2: RAG & AI Architect (เน้นขั้นตอนที่ 7-12: Online Pipeline)

เสร็จภายในวันที่ 5 ส่งงานต่อเพื่อน

คนนี้จะเป็นคนคุม "สมอง" ของระบบ ทำหน้าที่ดึง Context มาตอบคำถาม

- **Retrieval Logic:** ออกแบบการค้นหาแบบ Hybrid search หรือการจัดลำดับใหม่ (Re-rank) เพื่อให้ได้รีวิวที่ตรงกับความต้องการของผู้ใช้ที่สุด
- **Prompt Engineering:** เขียนคำสั่งให้ LLM (เช่น GPT-4 หรือ Claude) เข้าใจว่าต้องแนะนำหนังจากรีวิวที่ได้มาเท่านั้น และต้อง "ห้ามเดาเพิ่ม"
- **Citation System:** จัดการระบบอ้างอิงให้บอทระบุได้ว่าแนะนำหนังเรื่องนี้เพราะรีวิวไหนบน IMDB
- **Evaluation:** ทดสอบความถูกต้อง (Answer correctness) และความสอดคล้องกับเอกสาร (Groundedness) ตามเกณฑ์ประเมินในบทเรียน

[แม็ก]คนที่ 3: Full-stack Developer & UX Lead (เน้นหน้าตา App และ Presentation)

เสร็จภายในวันที่ 7 และ 8สรุป ประชุม&ส่งงาน

คนนี้จะเป็นคนทำ "ด่านหน้า" ให้ผู้ใช้งานเข้าถึงได้สะดวก

- **Web Application:** พัฒนาหน้าเว็บสำหรับ Chat โดยใช้ Stack ที่ถนัด (เช่น React หรือ Laravel ที่เคยใช้งาน) เพื่อเชื่อมต่อกับ API ของคนที่ 2
- **User Experience:** ออกแบบ UI ให้แสดงข้อมูลหนัง รูปภาพ และ Link กลับไปยัง IMDB เพื่อให้ดูน่าเชื่อถือ
- **Integration:** ดูแลภาพรวมให้ Flow ของข้อมูลตั้งแต่ผู้ใช้พิมพ์คำถามไปจนถึงได้คำตอบไหลลื่น
- **Presentation & Slide:** จัดทำสไลด์อธิบายหลักการทำงาน 12 ขั้นตอน เพื่อนำเสนอในชั้นเรียนตามที่โจทย์กำหนด

ข้อแนะนำเพิ่มเติมสำหรับโปรเจคนี้: เนื่องจากเอกสารเน้นเรื่อง **"ความเชื่อถือได้ของคำตอบ"** กลุ่มคุณควรระวังเรื่องการแบ่ง Chunk เพราะรีวิวหนังบางอันอาจจะยาวมากหรือสั้นเกินไป หากแบ่งไม่ดีบอทอาจจะดึงบริบทไปตอบผิดได้ครับ

วี ส่วนคนที่1

ข้อมูลที่นำมาใช้อ้างอิง
https://www.imdb.com/search/title/?title_type=feature&primary_language=th

แต่ละหนังจะเก็บข้อมูลเป็น json โดยมี format เป็น

```
{
        "title": "The Red Line",
        "imdb_id": "tt39744394",
        "release_year": "2026",
        "runtime": "2h 15m",
        "imdb_rating": "6.1",
        "synopsis": "Three women fall prey to a ruthless phone scam. To
settle the score, they'll take on the criminal network that stole their
money and their dignity.",
        "reviews": [
            "Rating: 4/10 | Unfortunately, บลา ยบลา",
]
    }
```

มีหนังไทยอยู่บนเว็บ IMDB = 1604 เรื่อง แต่จะมีหลายเรื่องที่ไม่มีทั้ง release_year, runtime,imdb_rating, synopsis เลยเติมข้อมูลที่อิงจาก letterboxd แบบ manual และจะตัดหนังที่ไม่มี imdb_rating ออก จะเหลือหนัง 939 เรื่อง

reviews จะมีเรื่องที่ไม่มีผู้ใช้รีวิวคอมเม้นเลย จะดักไว้เป็น No user reviews yet ไว้ก่อน

เลือกแบ่ง chunk เป็น 1 รีวิว = 1 chunk

อ๊อบ ส่วนคนที่2

อัปเดตไฟล์ และ code ใหม่ 7/4/2569
https://drive.google.com/drive/folders/1bt0zgwgI5GNo8w3O1h0ikhUmHMRlxkBx?usp=sharing >>> ลิงก์ดาวน์โหลดไฟล์

app	5/4/2569 15:14	Python Source File	16 KB
metadata	4/4/2569 13:06	JSON Source File	8,113 KB
movie_vectors	4/4/2569 14:33	INDEX File	7,574 KB
rag_demo	4/4/2569 13:06	Python Source File	6 KB
requirements	4/4/2569 14:46	Text Document	1 KB

1. app.py เป็นโค้ดส่วนคนที่ 2
2. metadata.json เป็น dataset imdb
3. movie_vetors.index เป็นไฟล์ที่วี คลีนแล้ว build ออกมาเป็นไฟล์ฐานข้อมูล
4. reg_demo โค้ดที่วี เขียน >>> เพื่อไว้ build ไฟล์ index
5. requirements >>> libary ที่ใช้ทั้งหมด

https://huggingface.co/organizations/chat-ai-movies-imdb/share/LgqThgGhsTGVxnIJvsabVjTzdOfQURzMxX >>> ลิงก์คำเชิญสำหรับ เข้า space group สมัครก่อนแล้วค่อยจ่อย ตั้งไว้ join auto ถ้าไม่ได้ทักนะ

https://huggingface.co/spaces/chat-ai-movies-imdb/imdb_chatbot >> link space สามารถเข้าดูไฟล์ได้

https://chat-ai-movies-imdb-imdb-chatbot.hf.space/ >> link direct url

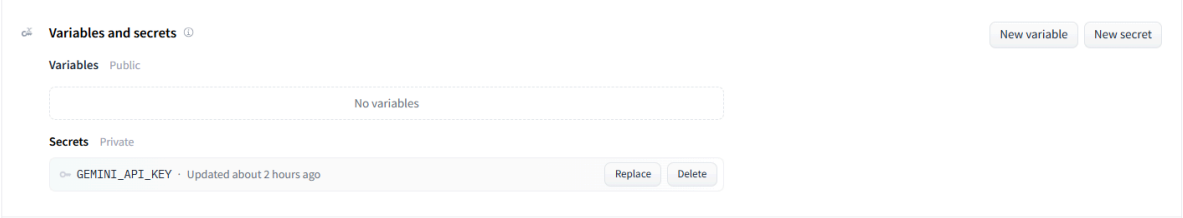
**นี่ใช้ Hugging Face นะ มัน deploy chat บนเว็บเป็นเป็นสาธารณะง่ายดี รันโค้ดบนเว็บเลย แล้วก็ใช้ Gradio -> gr.ChatInterface เป็น interface chat สำเร็จรูปที่โอเคดี แม็กจะใช้ react อย่างอื่นก็ได้นะไปปรับเปลี่ยนได้ตามสไตล์ตัวเอง/ app.py รันบน local ได้ แต่ แต่ต้องติดตั้ง [pip install gradio google-genai sentence-transformers faiss-cpu numpy] เพิ่มเอง ถ้าแม็กจะใช้ gradio ต่อก็แก้ในส่วน demo ตกแต่งเพิ่มให้ส่วนงานตรงตาม ux/ui ได้เลย (นี่แค่ให้ ai gen code เอามาทดสอบตอนคุยกับแชทเฉย ๆ)

```
# ========================================
# ส่วน UI (Gradio)
# ========================================
demo = gr.ChatInterface(
    fn=generate_rag_response,
    title="🎬 AI แนะนำหนังไทยจากรีวิว IMDb",
    description="ระบบแชทบอท RAG สำหรับแนะนำภาพยนตร์ไทย พัฒนาโดยใช้ FAISS และ Gemini API",
    examples=["อยากดูหนังผีหักมุม", "แนะนำหนังดลกคลายเครียดหน่อย", "ขอหนังรักวัยรุ่นซึ้งๆ"],
    cache_examples=False,
    fill_height=True
)
```

แต่ถ้ารันบน local ต้องเพิ่ม gemini key เองนะ

```
# 1. ตั้งค่า Gemini API ด้วย Client ตัวใหม่
gemini_client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))
```

ส่วนบนเว็บ hug ต้องเข้าไปที่ setting > varivles and secerts ต้องไป replace คีย์ใหม่ เวลา token มันหมด



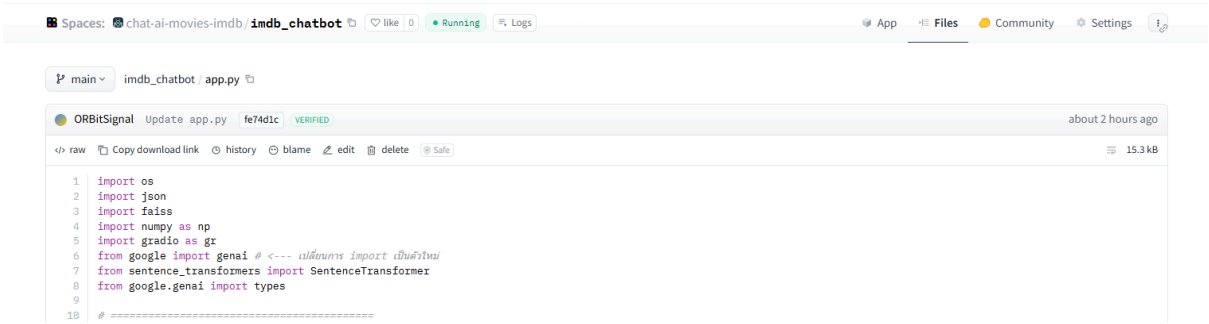
ให้ apikey ไว้เผื่อเอาไปเทสเวลาคุยกับแชทแล้วเต็ม limit:
ของวี

```
AIzaSyBLZZXCx-Sb_9XgX59gTSqQFrgrgwSrL4Y
```

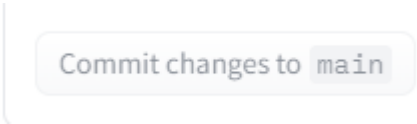
ของนี่

AIzaSyAfwB3gG5H4NJStd3bMh1guWNpooqolcg4
AIzaSyBin7X513rwGQLkp5gcT577HhIpW9d2XIc
AIzaSyDyswFYu7pIcZWJ_GbGC1woC0NZs-2lx0k
AIzaSyCAWkGbs3CdReueJrNJROWk96QQjgATQAE

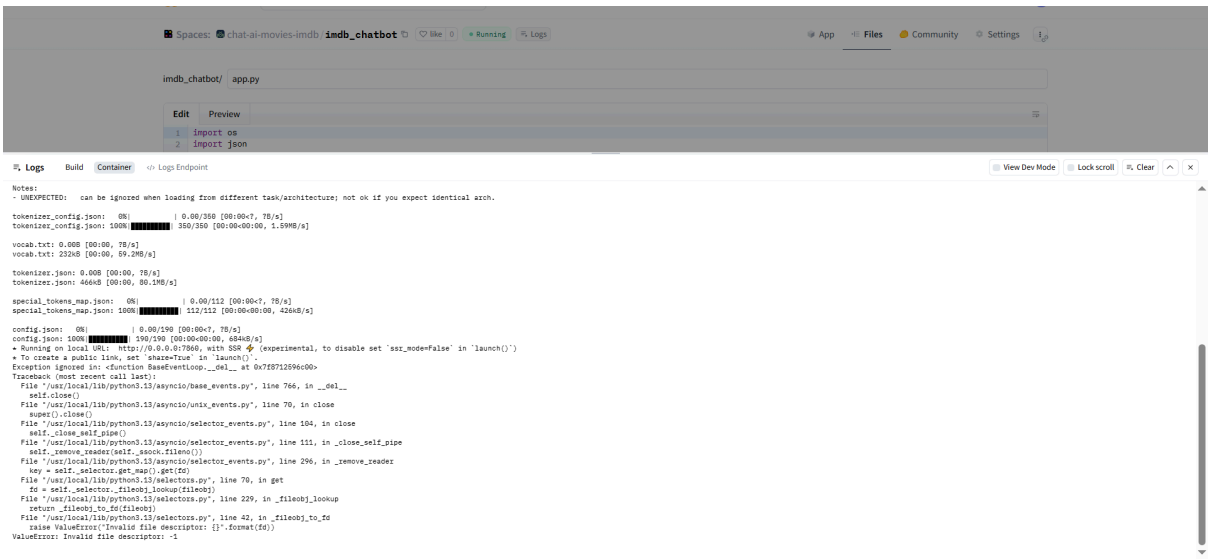
เวลาแก้ก็ไปที่ file แล้ว edit



เวลาแก้ไขเสร็จก็กด



ถ้าใช้งานได้ ด้านบนก็จะ running ถ้า error ก็เข้าไปดูที่ log นะ



~~note** rag_demo.py จาก all MiniLM L6 v2 ตัวที่รีใช้ รองรับภาษาอังกฤษเป็นหลัก ส่วนอีกตัวที่น่าใช้คือ paraphrase multilingual MiniLM L12 v2 **Embedding Model แบบ **Multilingual** (รองรับหลายภาษา)~~

```
2. แปลงความต้องการนั้นเป็น "คำหรือประโยคภาษาอังกฤษ (English Keyword/Query)" เพื่อนำไปค้นในฐานข้อมูลรีวิวภาพยนตร์
- หากคำถามใหม่เป็น "การถามต่อเนื่อง" ให้รวมบริบทแนวหนังเดิมเข้าไปด้วย แล้วแปล (เช่น คุยเรื่องหนังสยองขวัญอยู่แล้วถาม "มีเรื่องอื่นไหม" ให้แปลงเป็น "Other similar comedy movies")
- หากคำถามใหม่เป็น "การเปลี่ยนเรื่อง" ให้ทิ้งประวัติเก่าไปเลย และแปลเฉพาะคำถามใหม่ (เช่น "อยากดูหนังผี" แปลงเป็น "Horror movie" หรือ "Scary movie")
3. ตอบกลับมา "เฉพาะภาษาอังกฤษ" เท่านั้น ห้ามมีคำอธิบาย ห้ามมีเครื่องหมายคำพูดใดๆ เพิ่มเติมเด็ดขาด
```

~~เลยเขียน **prompt** ดัก เพราะเวลา **input** ด้วยภาษาไทยของ **user** ทำให้เวลา **ai** ปกติที่ไม่กำกับ **prompt** จะทำให้ตัวโมเดลภาษาที่ใช้เ~~
~~ดิมงง ค้นหาคีย์เวิร์ดในฐานข้อมูลไม่เจอ เลยใช้ **prompt** นี้แก้ปัญหาเฉพาะหน้าไปก่อนแทน~~

```python
# ใช้คำสั่งของไลบรารีตัวใหม่ในการเรียกใช้โมเดล
# --- STEP 12: Generate Answer + Citation (Smart & Safe) ---
try:
    # 1. เรียกใช้งาน Gemini พร้อมพยายามปลดล็อกเซ็นเซอร์
    response = gemini_client.models.generate_content(
        model='gemini-2.5-flash',
        contents=prompt,
        config=types.GenerateContentConfig(
            safety_settings=[
                types.SafetySetting(category="HARM_CATEGORY_DANGEROUS_CONTENT", threshold="BLOCK_NONE"),
                types.SafetySetting(category="HARM_CATEGORY_HARASSMENT", threshold="BLOCK_NONE"),
                types.SafetySetting(category="HARM_CATEGORY_HATE_SPEECH", threshold="BLOCK_NONE"),
                types.SafetySetting(category="HARM_CATEGORY_SEXUALLY_EXPLICIT", threshold="BLOCK_NONE")
            ]
        )
    )

    # 2. ดึงข้อความคำตอบ
    answer_text = response.text

    # 3. 🛡️ DEFENSIVE PROGRAMMING: ดักจับกรณี API ตรอปข้อความทิ้ง (ส่งค่าว่างมา)
    # ถ้า answer_text เป็น None หรือเป็นสตริงว่าง โค้ดจะหยุดทำงานตรงนี้และตอบผู้ใช้ทันที
    if not answer_text:
        return "⚠️ น้องฟีล์มขออภัยค่ะ! ดูเหมือนรีวิวหนังแนวนี้บางเรื่องจะมีเนื้อหาที่ระบบเซ็นเซอร์ขั้นสูงสุดของ Google บล็อกเอาไว้ (เช่น รีวิวที่มีคำบรรยายโหดร้ายหรือรุนแรงเกินไป) ลองถามหาแนวอื่นแทนนะคะ 🍿
```

ในส่วนโค้ดอันนี้คือ โค้ดที่ปลดฟิลเตอร์เซ็นเซอร์คำรุนแรง ที่บางคำวิจารณ์อาจจะมีพวก เลือดสาดบลา ๆ